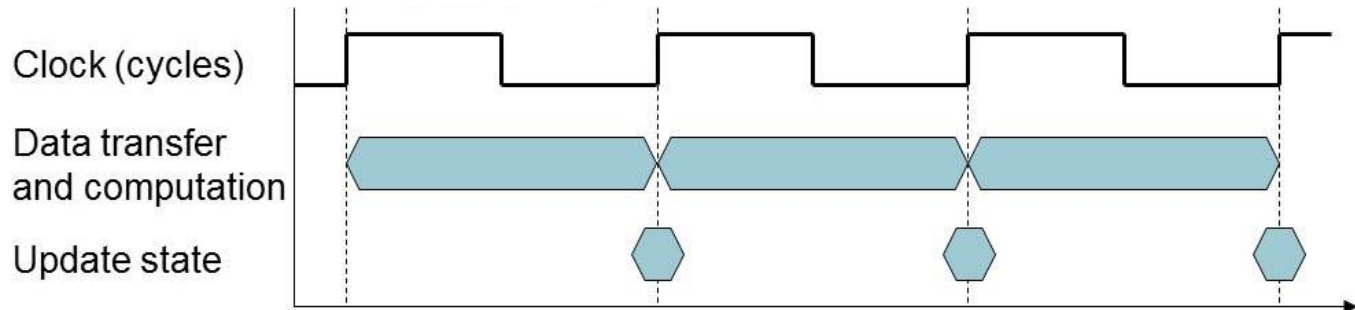


Lecture 2: Performance

CMPS 221 – Computer Organization and Design

Clocks

- A computer is driven by a **clock** that determines when events take place



- A clock **cycle** is a discrete time interval between two pulses of an oscillator
- A clock **period** is the duration of a clock cycle
- The **clock rate** or **frequency** is the number of clock cycles per second (inverse of the clock period)
 - Example: the Intel Core i7-8700K has a clock rate of 3.7GHz. What is its clock period?

Which has better performance?

- CPU₁: 2.4 GHz
- CPU₂: 3.8 GHz

Trick question!

Which has better performance?



It depends on the *performance metric* we care about

Which has better performance?



If we care about minimizing the time to transport one person from one place to another (i.e., **execution time** or **latency**)...

...the car

Which has better performance?



If we care about maximizing the number of people we can transport in a certain amount of time (i.e., **throughput**)...

...the bus

Which has better performance?



If we care about minimizing the energy it takes to transport people (i.e., **energy efficiency**)...

...bikes



Which has better *execution time*?

- CPU₁: 2.4 GHz
- CPU₂: 3.8 GHz

Still a trick question!

Components of Execution Time

$$\begin{aligned} \frac{\text{Seconds}}{\text{Program}} &= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Seconds}}{\text{Instruction}} \\ &= \underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Instruction Count}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}} \end{aligned}$$

Which has better *execution time*?

$$\begin{aligned} \frac{\text{Seconds}}{\text{Program}} &= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Seconds}}{\text{Instruction}} \\ &= \underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Instruction Count}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}} \end{aligned}$$

Instruction Count

CPI

Clock Rate ⁻¹

CPU₁

?

?

2.4 GHz

CPU₂

?

?

3.8 GHz

Cannot decide based on just the clock rate

Improving a CPU's Execution Time

$$\underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Instruction Count}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}}$$

Approaches to decreasing execution time involve decreasing one of these three components

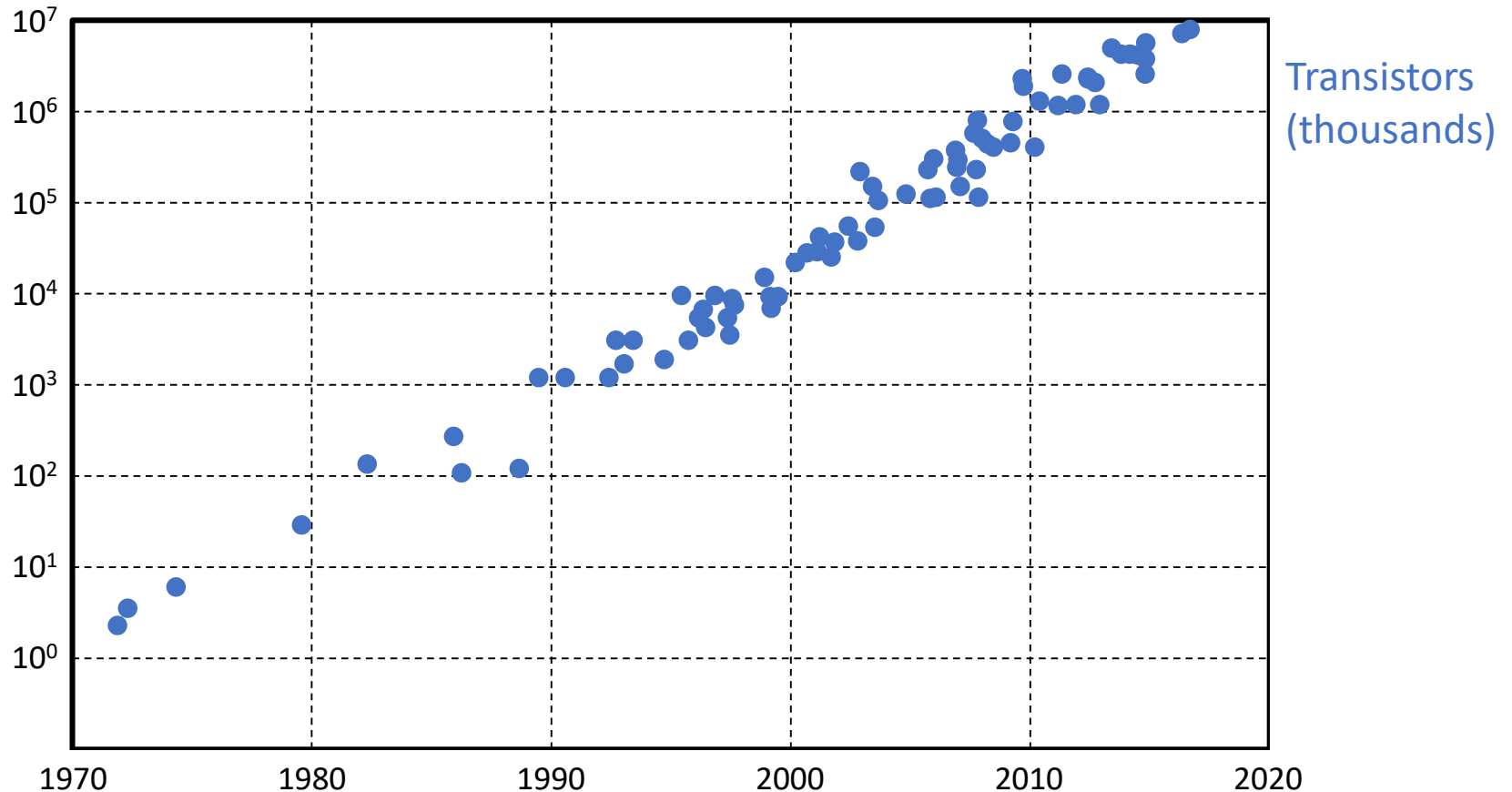
Improving a CPU's Execution Time

$$\underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Instruction Count}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}}$$

For a long time, improvements in circuits technology enabled driving processors at higher clock rates, improving execution time without the need for additional effort by software developers and computer architects

This trend was called the “free lunch”

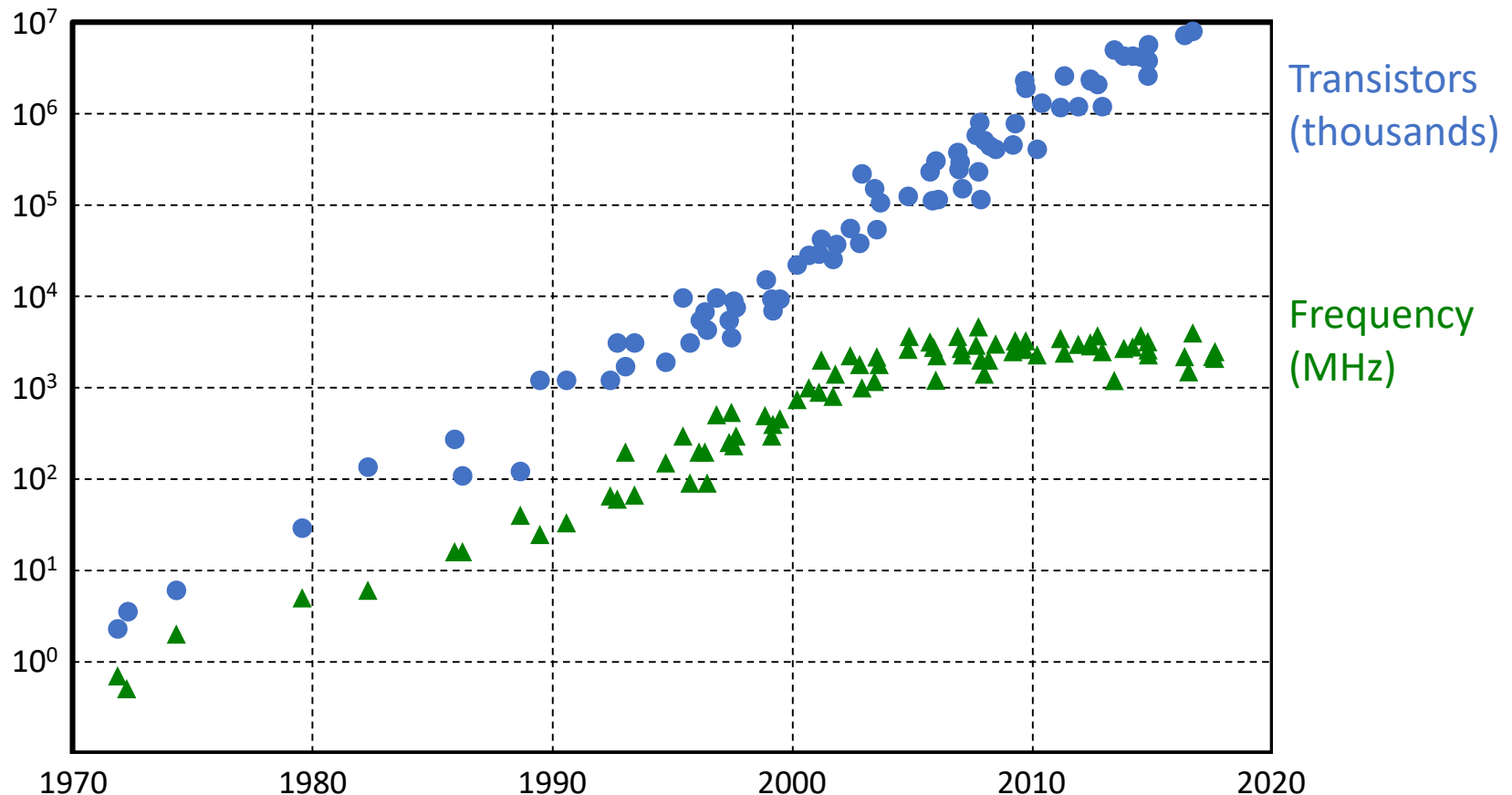
Moore's "Law"



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Moore's "Law" predicted that the number of **transistors** per unit area would double every 18-24 months

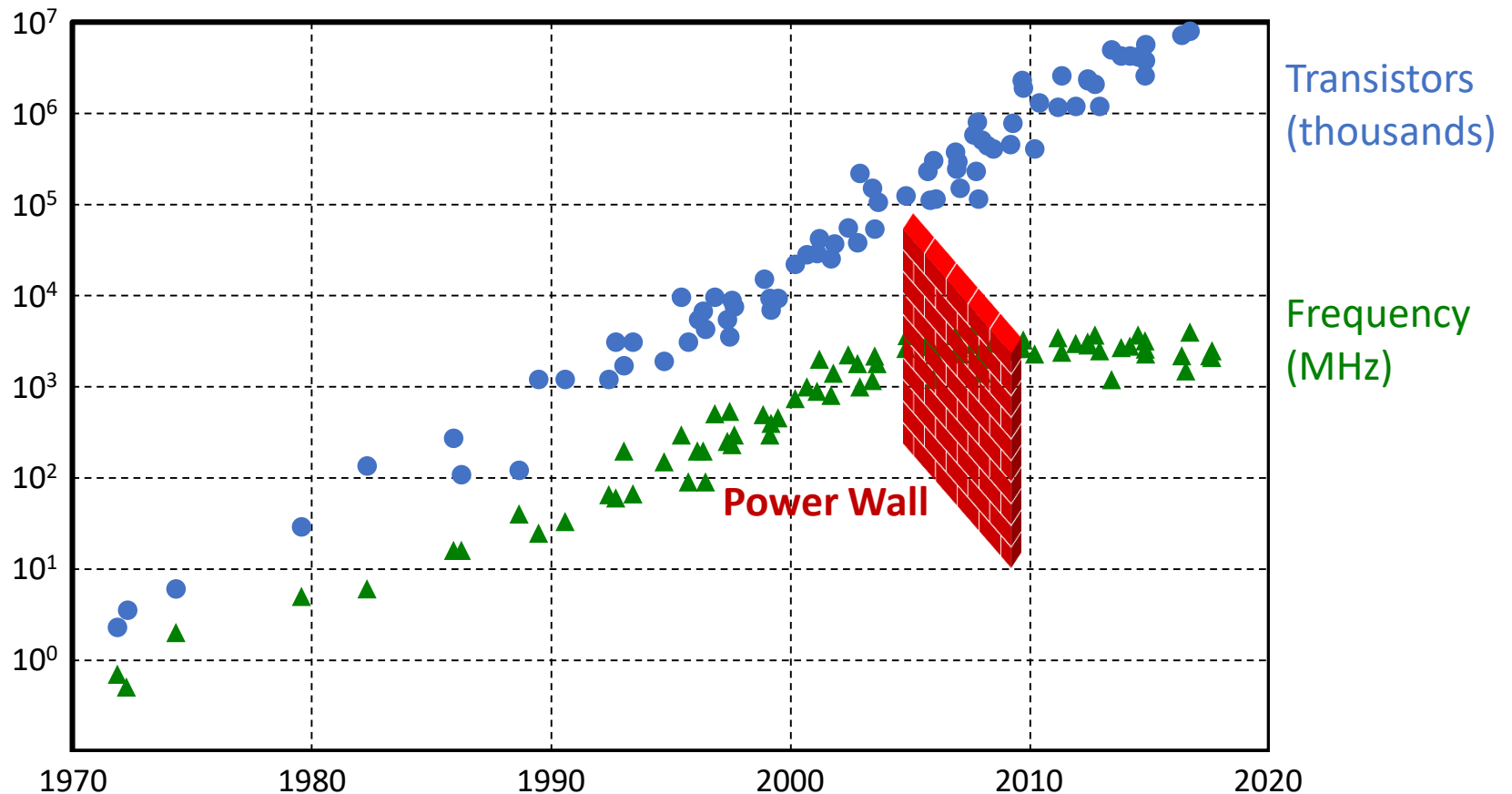
No More Free Lunch



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Processor **frequency** (clock rate) followed the same trend because smaller transistors can be switched faster... until around 2005.

Power Wall



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Around 2005, frequency stopped increasing due to the **Power Wall**

Power Breakdown

$$P \propto C V^2 f$$

(Power) *(Capacitance)* *(Voltage)* *(Frequency)*

Power Breakdown

$$\begin{array}{ccccccc} \uparrow P & \propto & C & V^2 & \uparrow f \\ \text{(Power)} & & \text{(Capacitance)} & \text{(Voltage)} & \text{(Frequency)} \end{array}$$

Increasing **frequency** increases **power** which dissipates more heat, requiring more support for cooling the chip

Power Breakdown

$$\begin{array}{ccccccc} \uparrow P & & \propto & & C & & \downarrow V^2 \uparrow f \\ \text{(Power)} & & & & \text{(Capacitance)} & & \text{(Voltage) (Frequency)} \end{array}$$

Historically, the increase in **power** was partially compensated for by a decrease in **voltage**, enabled by the decrease in transistor size
(over 20yrs, there was a 1,000x increase in frequency but only a 30x increase in power because voltage decreased by 5x)

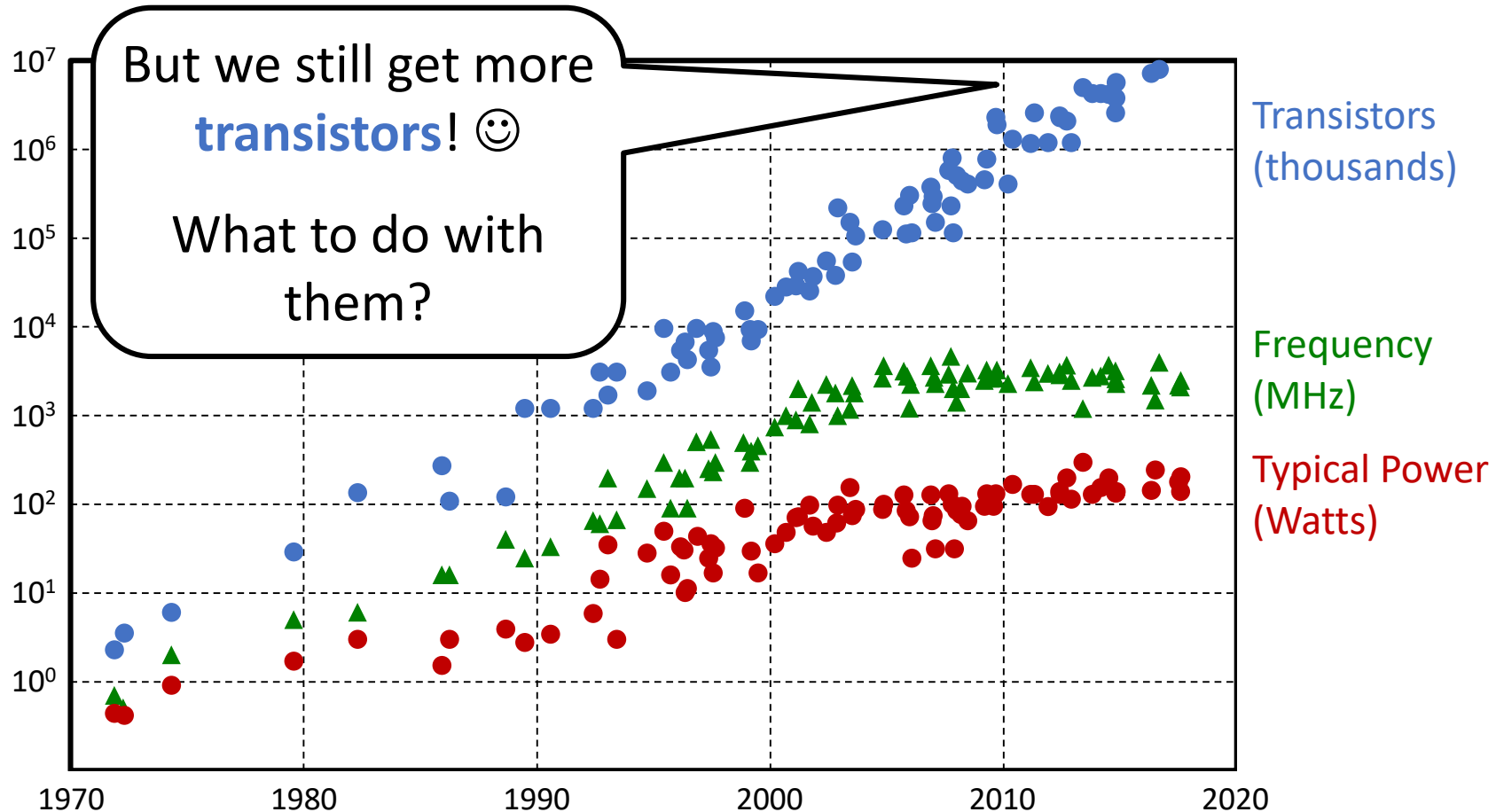
Power Breakdown

$$\begin{array}{c} \text{X} \\ \uparrow \\ P \\ \text{(Power)} \end{array} \propto \begin{array}{c} C \\ \text{(Capacitance)} \end{array} \begin{array}{c} \text{X} \\ \downarrow \\ V^2 \\ \text{(Voltage)} \end{array} \begin{array}{c} \text{X} \\ \uparrow \\ f \\ \text{(Frequency)} \end{array}$$

Today, **voltage** can no longer be decreased because it makes transistors unreliable and **power** can no longer be increased because we have reached the limit of what we can cool

therefore, **frequency** can no longer be increased.

Power Trend



Source: M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, C. Batten (1970-2010). K. Rupp (2010-2017).

Stagnation in **frequency** is associated with a stagnation in **power**

Where to invest transistors?

- Increase number of cores (or threads per core)
 - Improves throughput
- Make cores more advanced
 - Improves execution time
- Tradeoff between execution time and throughput

Improving a CPU's Execution Time

$$\underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Instruction Count}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}}$$

Computer architects have developed a wide variety of techniques for improving the number of instructions that can be executed each clock cycle

Techniques for Reducing CPI

- Pipelining (Chapter 4)
- Caching (Chapter 5)
- Speculative Execution (covered briefly)
- Out-of-order Execution (covered briefly)

Improving a CPU's Execution Time

$$\underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Instruction Count}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}}$$

Software plays a primary role in reducing a program's instruction count
(low complexity algorithms, powerful compiler optimizations)

Computer architecture also plays a role by providing special purpose
hardware for common operations (increasingly popular trend)

Pitfalls

$$\underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\text{Instruction Count}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\text{CPI}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}}$$

A CPU manufacturer increases the clock rate of their processor, decreasing the clock cycle duration.

As a result, some instructions that used to take 1 cycle to complete now require 2 cycles, increasing the overall CPI.

If CPI increase is disproportionate, execution time may increase.

Pitfalls

$$\underbrace{\frac{\text{Instructions}}{\text{Program}}}_{\substack{\downarrow \\ \text{Instruction Count}}} \times \underbrace{\frac{\text{Clock Cycles}}{\text{Instruction}}}_{\substack{\uparrow \\ \text{CPI}}} \times \underbrace{\frac{\text{Seconds}}{\text{Clock Cycle}}}_{\text{Clock Rate}^{-1}}$$

A CPU manufacturer creates a fused-multiply-add (FMA) instruction which is a common operation in linear algebra.

Assuming an add instruction takes 4 cycles and a multiply instruction takes 8 cycles, if the FMA instruction takes 12 cycles, then execution time does not improve.

Textbook Sections

- Some of the content in these slides corresponds to:
 - Textbook:
 - *Computer Organization and Design, 5th Edition by David Patterson and John Hennessy, Morgan Kaufmann, 2014.*
 - Sections:
 - 1.6, 1.7